

Problème 1 : ALGORITHME DE CHIFFREMENT RC4

A : Chiffrement d'une chaîne de caractères par l'algorithme RC4.

Dans cette partie on se propose de chiffrer (crypter) selon l'algorithme **RC4**, un texte sous forme d'une chaîne de caractères.

Déclarations globales préliminaires.

Dans toutes les questions de la **partie A**, on suppose avoir déjà déclaré les **constantes et variables globales** suivantes :

- **L1** et **L2** deux constantes entières déjà définies ($0 < L1 < 256$) et ($0 < L2 < 256$).
- **Claire** une chaîne de **L1** caractères contenant le texte à chiffrer (crypter).
- **Clef** une chaîne de **L2** caractères contenant la clé de chiffrement.
- **Chiffree** une chaîne de **L1** caractères destinée à contenir le texte après son chiffrement (cryptage).
- **K** une chaîne de **256** caractères
- **S** un tableau de **256** entiers (la table d'état)

Phase 1: Algorithme d'ordonnancement clé (Key Scheduling Algorithm (KSA))

Il s'agit dans cette **phase 1** de mettre en œuvre l'algorithme (KSA) en le décomposant en plusieurs fonctions.

❖ Question 1 : Initialisation du tableau S avec l'identité

- ✎ Ecrire la fonction de prototype **void identiteS()** ; qui permet d'initialiser les éléments du tableau **S** de telle sorte que chaque élément $S[i]$ ($0 \leq i < 256$) soit initialisé par la valeur de l'indice i ($S[0]=0, S[1]=1, \dots, S[i]=i, \dots, S[255]=255$).

❖ Question 2 : Initialisation du tableau K avec la clé

- ✎ Ecrire la fonction de prototype **void initialiserK()** ; qui permet d'initialiser la chaîne **K** avec la chaîne **Clef** comme suit :

$K[0]=Clef[0], K[1]=Clef[1], \dots, K[L2-1]=Clef[L2-1], K[L2]=Clef[0], K[L2+1]=Clef[1], \dots,$
 $K[255]=Clef[\dots]$

Exemple :

Soit la clé de longueur 3 ($L2=3$) suivante : **Clef="ABC"** alors le tableau **K** sera initialisé ainsi :
 $K[0]='A', K[1]='B', K[2]='C', K[3]='A', K[4]='B', K[5]='C', \dots, K[255]='A'$

❖ Question 3 : Permutation

- ✎ Ecrire une fonction de prototype **void permuterS(int i, int j)** ; qui permet de permuter $S[i]$ avec $S[j]$ (on suppose $0 \leq i < 256$ et $0 \leq j < 256$)

❖ Question 4 : Algorithme KSA

L'algorithme (KSA) est décrit comme suit :

- initialiser d'abord le tableau **S** (table d'état) avec l'identité.
- Initialiser le tableau **K** avec la chaîne Clef (représentant la clé de chiffrement).
- Mélanger les éléments de **S** de la façon suivante :
 - o Déclarer deux indices entiers **i** et **j** et initialiser **j** à 0.
 - o Pour tout **i** variant de 0 à 255, faire les instructions suivantes :
 - Instruction 1 : $j = (j + S[i] + K[i]) \text{ modulo } 256$
 - Instruction 2 : Permuter **S[i]** et **S[j]**
- Ecrire une fonction de prototype **void KSA()** ; qui réalise l'algorithme d'ordonnancement-clé (KSA), défini ci-dessus.

Phase 2: Pseudo Random Generator Algorithm (PRGA)

Pour réaliser l'algorithme (PRGA), on se propose de le découper en plusieurs fonctions.

❖ Question 5 : Décomposition binaire

- ✎ Ecrire une fonction de prototype **void decomposer (int bit[], int nombre)** ; qui met dans le tableau **bit** (1^{er} paramètre), la représentation binaire sur un octet (8 bits) de l'entier **nombre** (2^{ème} paramètre). On suppose ($0 \leq \text{nombre} < 256$).
- Dans cette décomposition, **bit[0]** contiendra le bit de poids faible (voir exemple).

Exemple :

Après l'appel de la fonction **decomposer(bit , 11)**, on aura :

$\text{bit}[0]=1, \text{bit}[1]=1, \text{bit}[2]=0, \text{bit}[3]=1, \text{bit}[4]=0, \text{bit}[5]=0, \text{bit}[6]=0, \text{bit}[7]=0$

❖ Question 6 : Puissance de deux

- ✎ En utilisant la **récurtivité**, définir la fonction d'entête : **int deuxPuissance(int n)** qui retourne la valeur 2^n (2 à la puissance n) (on suppose $n \geq 0$)

❖ Question 7 : valeur décimale d'un nombre en binaire

- ✎ Ecrire une fonction de prototype **int decimale(int bit [])** ; qui retourne la valeur décimale (base 10) de l'entier représenté en binaire par le tableau **bit** (le tableau **bit** est supposé être initialisé avec 8 entiers de valeurs 0 ou 1).

Exemple :

Soit le tableau **bit** initialisé ainsi :

$\text{bit}[0]=1, \text{bit}[1]=0, \text{bit}[2]=0, \text{bit}[3]=1, \text{bit}[4]=0, \text{bit}[5]=1, \text{bit}[6]=0, \text{bit}[7]=0$

Alors l'appel de la fonction **decimale(bit)** retourne le nombre **41**.

❖ Question 8 : Opérateur OU EXCLUSIF (XOR)

Soit x et y deux entiers tels que $(0 \leq x \leq 255)$ et $(0 \leq y \leq 255)$.

- ✎ Ecrire une fonction **int ouExclusif(int x, int y)** qui retourne le résultat de l'opération **x ouExclusif (XOR) y** (cette opération est effectuée bit à bit sur les représentations binaires de x et y selon les règles suivantes :

0 ouExclusif 0 vaut 0

0 ouExclusif 1 vaut 1

1 ouExclusif 0 vaut 1

1 ouExclusif 1 vaut 0

Remarque :

Il ne faut pas utiliser l'opérateur $\wedge$ (symbole de XOR en langage C).

Exemple : L'appel de la fonction `ouExclusif(5,6)` retourne 3 (En binaire $5 = 00000101$, $6 = 00000110$ et $5 \text{ OU EXCLUSIF } 6 = 00000011 = 3$).

❖ Question 9 : Algorithme PRGA

L'algorithme (PRGA) est décrit comme suit :

- Déclarer 3 entiers **i**, **j** et **a** puis initialiser **i** à 0 et initialiser **j** à 0
 - Déclarer un entier de nom **octet** (**octet** contiendra l'octet généré)
 - Pour **a** variant de 0 à (L-1), faire les instructions suivantes :
 - $i = (i+1) \text{ modulo } 256$
 - $j = (j + \text{Claire}[i] \text{ modulo } 256)$
 - Permuter $S[i]$ et $S[j]$
 - $\text{Octet} = S[(S[i] + S[j]) \text{ modulo } 256]$
 - $\text{Chiffree}[a] = (\text{Clef}[a] \text{ OU EXCLUSIF } (\text{octet}))$
- ✎ Ecrire la fonction **void PRGA()** qui permet de générer le texte chiffré en utilisant l'algorithme **PRGA** décrit ci-dessus pour chiffrer la chaîne **Claire** en utilisant la clé de chiffrement **Clef**, le texte sera mis dans la chaîne **Chiffree**.